

Relatório de Auditoria de Segurança — TrackTrash Smart Reverse Logistics

DATA: 04/05/2026

VERSÃO DO PROTOCOLO: v1.0-MVP (Sepolia Testnet)

AUDITORES: Análise automatizada via REDE ALVO: Ethereum Sepolia Testnet ferramentas formais

COMMIT ANALISADO: Branch main — solc 0.8.26

Escopo

Os seguintes contratos Solidity foram submetidos à análise:

Contrato	Arquivo	Linhas
ReverseLogistics	contracts/ReverseLogistics.sol	184
EcoToken	contracts/EcoToken.sol	—
EcoBadge	contracts/EcoBadge.sol	—
EcoStaking	contracts/EcoStaking.sol	—
EcoDAO	contracts/EcoDAO.sol	—
MockV3Aggregator	contracts/mocks/MockV3Aggregator.sol	—

A análise detalhada de achados concentra-se no contrato principal ReverseLogistics.sol, que contém a lógica crítica de negócio: registro

de devoluções, cobrança de taxa em ETH, mint de tokens ERC-20 e NFTs ERC-721.

Metodologia

Ferramentas utilizadas:

- **Slither v0.11.5** (análise estática) — ferramenta desenvolvida pela Trail of Bits.
- **Mythril** (execução simbólica) — tentativas de instalação documentadas abaixo.
- **Hardhat** (testes locais e PoC) — suite de testes local.

Comando executado (Slither):

```
# Instalação do compilador Solidity via solc-select
solc-select install 0.8.26 && solc-select use 0.8.26

# Análise estática
slither contracts/ReverseLogistics.sol --solc-remaps "@openzeppelin/
=node_modules/@openzeppelin/"

# Saída: 101 detectores executados · 20 resultados encontrados
```

Tentativas de instalação do Mythril:

Tentativa	Comando	Resultado
1 — Global	<code>pip3 install mythril</code>	✗ Failed to build pyethash
2 — Venv	<code>pip install mythril==0.23.22</code>	✗ Microsoft Visual C++ 14.0+ required

Diagnóstico definitivo: a dependência pyethash requer Microsoft C++ Build Tools 14.0+, não instalado no ambiente. O bloqueador é de infraestrutura.

Achados

Achado 1 — SLI-01

Vulnerabilidade: Ausência de Verificação de Freshness do Oráculo de Preço

Ferramenta: Slither — Detector unused-return

Severidade: Low

Impacto: Contrato aceita dados desatualizados do AggregatorV3Interface.

Probabilidade: Baixa

Descrição Técnica:

Em ReverseLogistics.sol, a função `_ethUsdPrice()` ignora os campos de controle de freshness recomendados pela Chainlink:

```
// contracts/ReverseLogistics.sol - linha 178
(, int256 answer, , uint256 updatedAt, ) = priceFeed.latestRoundData();
// ^roundId    ^startedAt    ^answeredInRound    ← todos ignorados
```

Achado 2 — SLI-02

Vulnerabilidade: Violação do Padrão CEI (Checks-Effects-Interactions)

Severidade: Low

Impacto: Chamadas externas realizadas antes de escritas no estado.

Probabilidade: Baixa (mitigado pelo nonReentrant)

Descrição Técnica:

```
// INTERACTION 1
ecoToken.mint(msg.sender, rewardAmount);

// EFFECT (Executado após as interações)
returnsById[returnId] = ReturnRecord({...});
```

Achado 3 — SLI-03

Vulnerabilidade: Pragma de Versão Solidity Excessivamente Permissivo

Severidade: Informational

```
pragma solidity ^0.8.20; // ← permite 0.8.20 a 0.8.x
```

Tabela Resumo

ID	Vulnerabilidade	Severidade	Impacto
SLI-01	Ausência de verificação de freshness	Low	Cálculo incorreto de taxa
SLI-02	Violação do padrão CEI	Low	Superfície de reentrância
SLI-03	Pragma permissivo	Informational	Bugs conhecidos

Recomendações

SLI-01 — Adicionar verificação de freshness

```
require(answeredInRound ≥ roundId, "Incomplete round");
require(block.timestamp - updatedAt ≤ MAX_ORACLE_STALENESS, "Price too old");
```

SLI-03 — Fixar versão

```
pragma solidity 0.8.26;
```

Conclusão

A auditoria não identificou vulnerabilidades críticas. Os achados são classificados como **Low** ou **Informational**. O protocolo demonstra maturidade sólida para um MVP acadêmico.

Repositório do Projeto: github.com/leomatiazzz/tracktrash-web3